

# 300

## Years Later — studio guide

Build a realistic 3D co-op game in Unreal Engine 5.8 with an autonomous AI, from A to Z: setup, code, tests, compliance, media, cinematic landing page, launch.

Edition of 2026-10-07 · CC BY 4.0 licence

# Contents

1. Read first
2. Quick start
3. A to z
4. Installation
5. Time and cost
6. Security and privacy
7. Customise
8. Other ais
9. Faq troubleshooting
10. Diagrams
11. Screenshots
12. Game images

# Read first

**This guide contains no game image** until a render has been produced in Unreal Engine and approved by a human. The screenshots shown are real: the reference landing page and the outputs of the kit's tools. Diagrams explain how things work; they do not depict the game. Diagram labels are in French.

The target is a high-quality realistic indie title, not a production of several hundred million. Budgets and durations are detailed in the "Time and cost" guide.

# Quick start (10 minutes)

 [Version française](#)

## What you get

An **AI-driven game studio**: the complete design of a realistic 3D co-op game (*300 Years Later*), a legal pack, a cinematic landing page, tests, and above all **skills** that let Claude Code (or another AI) build the game in Unreal Engine 5.8 autonomously from A to Z, only coming to you for the decisions that are yours.

## What you need

- A Windows 10/11 PC with a recent GPU (RTX 3070/4070 or better), 32 GB RAM, 300 GB free — details in [02\\_INSTALLATION.md](#).
- Claude Code (or another coding agent) and an Epic Games account.
- Realistic time and budget: read [03\\_TIME\\_AND\\_COST.md](#) before you start.

## One message to your AI

```
git clone <THIS-REPO-URL> 300-years-later && cd 300-years-later
claude          # or gemini, kimi, codex, cursor, aider...
```

Then paste:

Read AGENTS.md, then apply the game-studio skill. Work autonomously from A to Z following studio.config.yaml; only come to me at hard stops, batching your questions with your recommendation. Reply in English. *(optional)* Extra instructions: ...

Prefer to stay in control? Set `autonomy.level: guided` in `studio.config.yaml` (the AI asks before each step).  
Want it to install tools without asking too? `full`.

## Configure the AI once: `studio.config.yaml`

| Key                                             | Effect                                                                      |
|-------------------------------------------------|-----------------------------------------------------------------------------|
| <code>autonomy.level</code>                     | <code>guided</code> · <code>autonomous</code> (default) · <code>full</code> |
| <code>autonomy.max_questions_per_session</code> | maximum questions per session (default 3)                                   |
| <code>project.languages</code>                  | game and public-text languages (default fr, en)                             |
| <code>defaults.*</code>                         | team scenario, monthly asset budget, art direction, landing mode...         |
| <code>extra_instructions</code>                 | your free-form instructions, applied every session                          |

Your message always overrides this file.

## Then

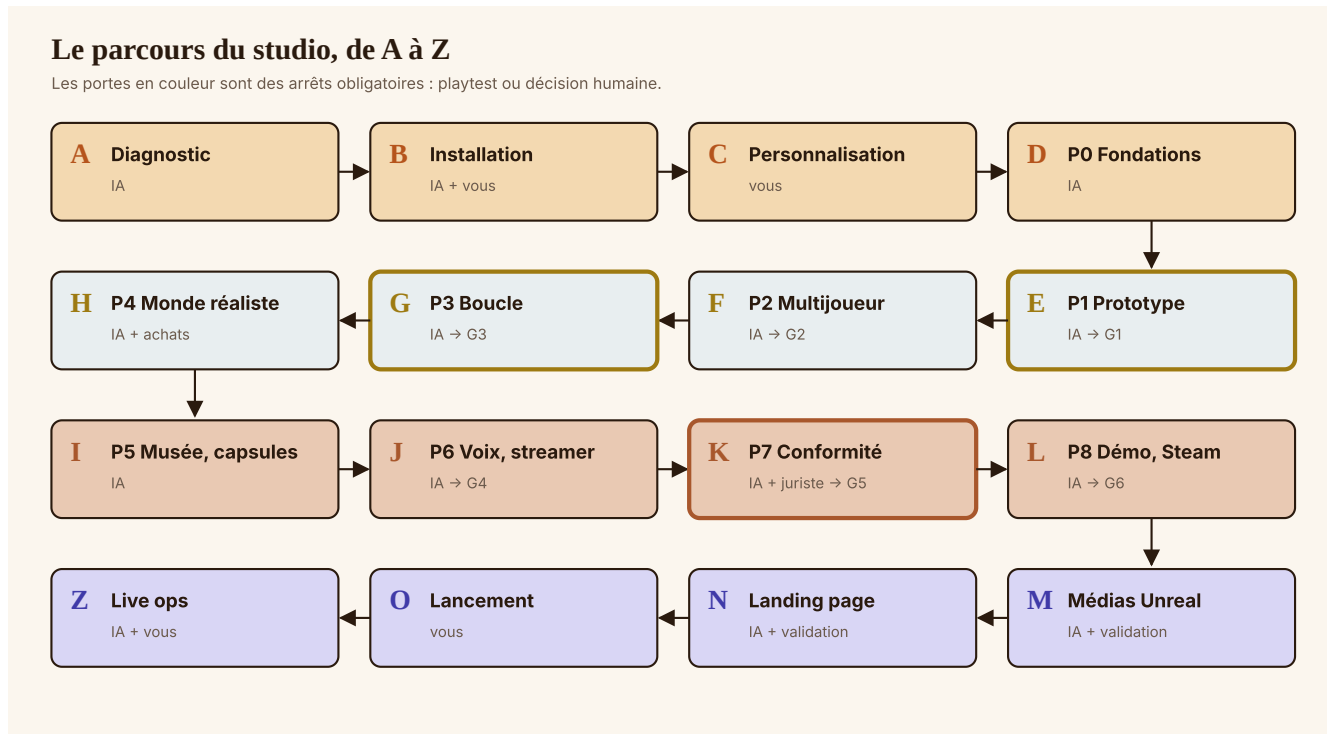
The AI moves forward on its own and keeps three files up to date: `STUDIO_STATE.md` (where we are), `DECISIONS.md` (what it decided alone, reversible), `QUESTIONS.md` (what it needs from you, with its

recommendation — reply "ok" to accept). Hard stops (accounts, Unreal installation, purchases, publishing...) are described in the [A to Z guide](#).

# A to Z guide

 [Version française](#)

This guide describes the full journey, from the cloned repository to the game in Early Access, as driven by the game-studio skill. For each step: **who** does what, **what you check**, and the **indicative duration** ("small studio" scenario in [03\\_TIME\\_AND\\_COST.md](#)).



## Roles

- **AI:** Claude Code (or another agent) applying the skills, in autonomous mode by default (`studio.config.yaml`).
- **You:** the project owner, who decides, buys, approves and publishes.
- **Pro:** an external professional (lawyer, artist, composer...).

## A. Diagnosis — 1 hour

**AI** runs `tools/doctor.py`, checks the hardware, reads `STUDIO_STATE.md`. **You** read the report. If the PC is not powerful enough, the AI can still advance the design, the landing page and the legal pack.

## B. Setup — 1 to 3 days (long downloads)

**AI** shows the installation plan (`install_windows.ps1 -Plan`), then runs it after one global consent (no question in full mode). Epic's plugin and frontend-design are already declared in `.claude/settings.json`. **You** create the Epic account, install Unreal Engine 5.8 from the Epic Games Launcher, accept the plugin install in Claude Code, enable the MCP server in the editor. Details: [02\\_INSTALLATION.md](#). **Check:** `doctor.py` all green; the AI lists the actors of the open map through MCP.

## C. Customisation — 1 day

**AI** reads `studio.config.yaml`; **you** only step in if the file is empty or to change the public title, languages, platforms ([05\\_CUSTOMISE.md](#)). **Pro** (or you): trademark clearance (`legal/21_marque-pi.md`). **Check**: `STUDIO_STATE.md` up to date.

## D. Foundations (P0) — 2-3 weeks

**AI** creates the Unreal C++ project (`game/`), the `TemporalCore` and `TemporalValley` modules, Windows CI, architecture decision records (ADR). **You** create the **private** repository for binary content (`game/Content/`), and install a GitHub Windows runner if you want the game CI. **Check**: green build, green automated tests.

## E. Fun prototype (P1) — 4-8 weeks · Gate G1

**AI** codes the temporal core (tests first) then a playable prototype: planting a seed in year 0 grows a tree in year 300. It runs bot playtests, self-reviews against the design pillars, then queues a human playtest in `QUESTIONS.md` (non-blocking in autonomous mode). **You** run that playtest with 5 people (kit provided: `legal/14_playtest-consentement.md`, grid in `docs/design/51`). **Decision**: if nobody laughs or is surprised within 10 minutes, iterate; two failures = pivot.

## F. Multiplayer (P2) — 6-10 weeks · Gate G2

**AI**: Steam sessions (test AppID), 4 players, 4 eras, ghosts, integrity check. **You**: a 4-player test session with friends. **Check**: Gauntlet test "4 identical hashes".

## G. Game loop (P3) — 8-12 weeks · Gate G3

**AI**: rounds, coffee break, rotation, contracts, paradoxes, hazards, HUD. **You**: playtest with 8 people; trademark filing; Steam page (Steam Direct); capsule commission. **Check**: "I'd play again" score  $\geq 4/5$ .

## H. Realistic world and content (P4) — 6-12 months

**AI** prepares the asset **shopping list** (Fab, MetaHuman) with licences and costs, and starts with free Epic content so work never waits. **You** buy and add the assets; commission the creatures from an artist; approve the art direction. **AI** builds the landscape, eras, procedural vegetation, ageing prefabs, and measures performance. **Check**: 10 playable world seeds; performance budgets met on the 3 profiles.

## I. Clip machine (P5) — 6-10 weeks

End-of-match museum, shareable capsules, solo mode, tutorial, photo mode.

## J. Voice, streamer, saboteur (P6) — 6-8 weeks · Gate G4 (alpha)

**AI**: cross-era voice, anti-harassment tools, streamer mode (off by default). **Check**: "no audio on disk" test, 3 streamers in closed test.

## K. Compliance (P7) — 8-12 weeks · Gate G5 (beta)

**AI** updates the `legal/` pack from the real game (FR + EN) and prepares the lawyer's packet. **Pro**: legal review. **You**: integrate the corrections. **AI** + **you**: full accessibility, localisation, pseudo-localisation.

## L. Demo and Steam (P8) — 8-12 weeks · Gate G6

Demo build, achievements, leaderboards, Cloud, profiling, Shipping build. Steam content survey (including the AI section).

## M. Media — continuous from G3

**AI** renders the shots of `data/shotlist.json` **inside Unreal** (Movie Render Graph) and edits the trailers (Sequencer + Remotion). **You** approve each file in `media/APPROVALS.md`. Nothing unapproved is published.

## N. Landing page — 1-2 weeks, then updates

**AI** builds the **cinematic** landing (`docs/design/34_LANDING_CINEMATIQUE.md`): animated typographic mode while no render is approved, then an automatic switch to the video hero and the scroll-driven 4-era sequence once shots L00–L04 are approved; it runs the tests (Robot Framework, Lighthouse, k6, OWASP ZAP). **You** approve the copy and trigger publication (`manual/pages.yml` workflow).

## O. Launch — 4-8 weeks

Steam Next Fest, creator programme (`legal/12_politique-createurs.md`), communication. **You** publish; the AI prepares.

## Z. After launch

Fixes within 72 h for severe issues, regular updates, seasons (`docs/design/71_LIVE_OPS.md`).

---

**During hard stops**, the AI prepares everything (click list, files) and keeps independent tracks moving (landing, legal, data, Remotion).

**Reminder:** the AI never buys, publishes, signs, or chooses the name or price on its own. See [04\\_SECURITY\\_AND\\_PRIVACY.md](#).

# Studio workstation installation

 [Version française](#)

## 1. Hardware

|         | Practical minimum                 | Recommended                              |
|---------|-----------------------------------|------------------------------------------|
| OS      | Windows 10 64-bit                 | Windows 11 64-bit                        |
| CPU     | 8 cores                           | 12-16 cores (C++ and shader compilation) |
| Memory  | 16 GB (slow)                      | 32-64 GB                                 |
| GPU     | 8 GB VRAM (RTX 2070 / RX 6600 XT) | RTX 3070/4070 or better, 12 GB+          |
| Storage | 300 GB SSD                        | 1 TB NVMe                                |
| Network | fibre advised                     | engine + assets = 100-200 GB to download |

macOS works for Unreal (Epic Games Launcher + Xcode) but the project targets Windows first. Linux is used for side tools (landing, tests).

## 2. Software

The studio-setup skill installs everything with winget:

```
powershell -ExecutionPolicy Bypass -File .claude/skills/studio-setup/scripts/install_windows.ps1
-Plan # show the plan
powershell -ExecutionPolicy Bypass -File .claude/skills/studio-setup/scripts/install_windows.ps1
# install
```

| Tool                                                       | Why                                        |
|------------------------------------------------------------|--------------------------------------------|
| Epic Games Launcher + <b>Unreal Engine 5.8</b>             | engine (installed in the launcher, by you) |
| Visual Studio 2022 (C++ game + desktop workloads) or Rider | compilation                                |
| .NET 8, Git, Git LFS, GitHub CLI, Python 3.12              | build tools and kit tools                  |
| Node LTS, ffmpeg                                           | landing page, video                        |
| Blender LTS                                                | asset touch-ups                            |
| Robot Framework, k6, gitleaks, semgrep                     | tests and security                         |

## 3. Connect Claude Code to Unreal (Epic's official plugin)

1. The plugin is already declared in `.claude/settings.json` (`enabledPlugins`): Claude Code offers to install it when you open the folder. Otherwise: `/plugin install unreal-engine-skills-for-claude-code@claude-plugins-official` (and `frontend-design@claude-plugins-official` for the landing).
2. In the Unreal editor: *Edit > Plugins*, enable **Model Context Protocol** and **AllToolsets**, restart.
3. Editor console: `ModelContextProtocol.StartServer`
4. On Windows, Git Bash (installed with Git) must be on the PATH.
5. Test: ask Claude "list the actors in the open map".

⚠ This plugin gives the AI broad access to the editor, including running Python. Always work on a Git branch and commit before each batch of operations.

## 4. Variables and folders

```
setx UE_ROOT "C:\Program Files\Epic Games\UE_5.8"
```

Create a **private** repository for game/Content/ (Git LFS or Perforce). Never publish this folder.

## 5. Check

```
py tools\doctor.py
```

Every relevant line must be ✓.

# Time, cost and consumption

 [Version française](#)

Order-of-magnitude estimates as of 7 October 2026, for a **realistic 3D** co-op game in Unreal Engine 5.8 as described in docs/design/. They are for deciding, not promising. Measure your real figures at each gate and adjust.

## 1. Three scenarios

|                              | A — Solo + AI                                                    | B — Small studio                                                     | C — Small team                                                                       |
|------------------------------|------------------------------------------------------------------|----------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Team                         | project owner (part-time) + Claude Code + occasional freelancers | full-time owner + 2-3 regular freelancers (3D art, animation, sound) | 5-8 people (gameplay programmer, tech artist, 2 artists, animator, producer/QA) + AI |
| Time to Early Access         | 30-48 months                                                     | 18-30 months                                                         | 15-24 months                                                                         |
| Budget excluding owner's pay | ≈ €40,000 – 80,000                                               | ≈ €150,000 – 300,000                                                 | ≈ €500,000 – 1.2M                                                                    |
| Achievable visual level      | realistic thanks to purchased assets, mostly library animations  | consistent realism, custom creatures and NPCs                        | polished realism, performance capture, richer content                                |
| Main risk                    | duration and scope                                               | cash flow                                                            | monthly fixed cost                                                                   |

## 2. Cost items (ranges)

| Item                                         | Scenario A        | Scenario C         | Notes                                                                 |
|----------------------------------------------|-------------------|--------------------|-----------------------------------------------------------------------|
| Workstation (if needed)                      | €2,000 – 3,500    | €2,500 × people    | RTX 4070+ GPU, 64 GB advised                                          |
| Unreal Engine                                | €0                | €0                 | 5 % above \$1M lifetime gross revenue (Epic Games Store sales exempt) |
| Fab / Megascans assets                       | €1,000 – 6,000    | €8,000 – 30,000    | Megascans paid since 2025                                             |
| Creatures (sculpt, groom, rig)               | €3,000 – 10,000   | €15,000 – 40,000   | Bouloche, goats, Chronomites                                          |
| Facial / body capture                        | €0 – 3,000        | €15,000 – 60,000   | MetaHuman Animator (video) vs mocap studio                            |
| Music (6 tracks + stingers)                  | €3,000 – 8,000    | €10,000 – 30,000   | rights assignment included                                            |
| Additional sound design                      | €1,000 – 4,000    | €8,000 – 25,000    |                                                                       |
| Steam capsule, logo, illustrations           | €1,500 – 4,000    | €5,000 – 15,000    |                                                                       |
| Trailer editing and grading                  | €1,000 – 3,000    | €5,000 – 20,000    |                                                                       |
| Localisation (≈ 15,000 words × 7 languages)  | €8,000 – 16,000   | €12,000 – 25,000   | professional per-word rates; native review                            |
| Legal (terms, privacy, contracts, DPIA)      | €2,000 – 6,000    | €8,000 – 20,000    |                                                                       |
| Trademark (EU, 1-3 classes)                  | €850 – 1,500      | €2,000 – 5,000     | + international if needed                                             |
| Accountant, professional liability insurance | €1,500 – 3,500/yr | €4,000 – 10,000/yr |                                                                       |
| Steam Direct                                 | ≈ \$100           | ≈ \$100            | per game                                                              |
| Playtests and external QA                    | €500 – 3,000      | €10,000 – 40,000   |                                                                       |
| Marketing (events, PR, ads)                  | €2,000 – 10,000   | €30,000 – 150,000  |                                                                       |
| AI assistant subscription                    | depends on plan   | plan × people      | see plans on support.claude.com                                       |
| Team salaries                                | —                 | €400,000 – 900,000 | main item of scenario C                                               |

Possible support in France: video game tax credit (strict conditions), CNC video game fund, Bpifrance, regional grants ([legal/20\\_fiscalite-redevances.md](#)). Elsewhere, check your national and regional schemes.

### 3. Calendar by phase (scenario B)

| Phase                                      | Duration    | Gate      | What happens                                  |
|--------------------------------------------|-------------|-----------|-----------------------------------------------|
| A-C Diagnosis, setup, customisation        | 1-2 weeks   | —         | workstation ready, title chosen               |
| P0 Foundations                             | 2-3 weeks   | —         | Unreal project, CI, ADRs                      |
| P1 Fun prototype                           | 4-8 weeks   | <b>G1</b> | playable seed → tree, 5 testers               |
| P2 Multiplayer                             | 6-10 weeks  | G2        | 4 players online without desync               |
| P3 Game loop                               | 8-12 weeks  | <b>G3</b> | full match, Steam page, first clips           |
| P4 Realistic world and content             | 6-12 months | —         | the longest: assets, PCG, prefabs, MetaHumans |
| P5 Clip machine                            | 6-10 weeks  | —         | museum, capsules, solo                        |
| P6 Voice, streamer, saboteur               | 6-8 weeks   | G4        | alpha                                         |
| P7 Compliance, accessibility, localisation | 8-12 weeks  | <b>G5</b> | beta, legal review                            |
| P8 Demo and Early Access                   | 8-12 weeks  | G6        | demo, Next Fest, launch                       |

## 4. AI consumption (order of magnitude)

- **Agent time:** a code phase typically takes 20 to 80 Claude Code work sessions, from 30 minutes to a few hours each; P4 is the hungriest (many editor operations via MCP).
- **Tokens:** highly variable with project size, caching and the number of editor operations. Expect several million tokens per intensive working day. Measure your real consumption (your tool's cost/usage command) from P0 and record it in `STUDIO_STATE.md` to extrapolate.
- **Savings:** one phase per session, plan mode first, small PRs, targeted tests, avoid re-reading the whole repository each session (`game-studio` reads `STUDIO_STATE.md` first).
- **Incompressible human time:** PR reviews ( $\approx 20\text{--}30\%$  of agent time, lower in autonomous mode), playtests, asset purchases, art direction, media and copy approval, gate decisions.

## 5. What blows up estimates

Scope (adding an era, a biome, a mode), the quality of realistic humans, motion capture, late localisation, and no early playtests. The cut rule of `docs/design/50_PRODUCTION_PLAN.md` §6 applies before any budget extension.

# Security and privacy: how the kit protects you

 [Version française](#)

### Modèle de sécurité du kit

#### L'IA peut

- ✓ lire et écrire dans le dépôt
- ✓ installer depuis les sources officielles, après accord
- ✓ compiler, tester, mesurer
- ✓ piloter l'éditeur Unreal sur une branche
- ✓ préparer documents, médias, landing page
- ✓ proposer achats, textes et décisions

#### L'IA ne fait jamais seule

- × acheter, créer un compte, saisir un identifiant
- × publier (GitHub public, Steam, réseaux, site)
- × signer, déposer une marque, fixer le prix
- × lire ~/.ssh, .env, trousseaux, fichiers perso
- × ajouter traceur, cookie, CDN, télémétrie
- × publier Content/ ou des assets sous licence

Garde-fous : .claude/settings.json (deny/ask) · tools/privacy\_scan.py · hook pré-commit · CI (gitleaks, semgrep, ZAP) · media/APPROVALS.md

Votre liste locale de termes interdits (~/.config/300yl/denylist.txt) n'est jamais versionnée.

## 1. What the AI may do alone

Read and write **inside the repository**, install tools **from official sources** (after one global consent in autonomous mode, without asking in full mode), compile, test, drive the Unreal editor through MCP on a branch, prepare documents and media, take reversible decisions logged in DECISIONS.md.

## 2. What the AI never does alone (hard stops, at every autonomy level)

| Action                                                   | Why                                           |
|----------------------------------------------------------|-----------------------------------------------|
| Buy (assets, Steam Direct, contractors)                  | your money                                    |
| Create an account, type in credentials                   | your identity                                 |
| Publish (public GitHub, Steam, social networks, website) | your responsibility as publisher              |
| Sign, file a trademark                                   | legal commitment                              |
| Choose the title, price, public copy                     | publisher decisions                           |
| Read your personal files, SSH keys, .env, keychains      | deny rules in .claude/settings.json           |
| Send your data to a third-party service                  | no network call outside official sources      |
| Add a tracker, cookie, CDN                               | forbidden and detected by privacy_scan.py     |
| Publish game/Content/ or Epic/Fab/MetaHuman assets       | forbidden by licences, blocked by CI          |
| Publish an unapproved media file                         | media/APPROVALS.md + check_media_approvals.py |

## 3. Protect your personal data

1. Create ~/.config/300yl/denylist.txt (outside the repo) with your first and last name, email, handle, employer, city. tools/privacy\_scan.py will reject any file containing them.

2. Configure Git with GitHub's anonymous address: `git config user.email "<id>+<handle>@users.noreply.github.com"`.
3. Enable the pre-commit check: `git config core.hooksPath .githooks`.
4. Use an email address dedicated to the project for all public texts (terms, legal notice, press).
5. Strip media metadata before publishing (`exiftool -all=`) — the `privacy-guard` skill does it.

## 4. Check for yourself

```
python3 tools/privacy_scan.py
cat .claude/settings.json
git ls-files | grep -Ei '\.(uasset|umap|pak)$|^game/Content/' # must be empty
```

## 5. Players

The game applies privacy by default (legal/03, legal/09): voice never recorded, opt-in statistics, friends-only sessions by default, no in-app purchase.

# Customise the project (or make it your own game)

 [Version française](#)

Everything is designed to be changed. Licence: MIT code, CC BY 4.0 content (credit "The 300 Years Later contributors"). The title and trademark are not covered by the licence.

## 1. Tell the AI what you want

- **Once:** extra\_instructions in studio.config.yaml (e.g. "English first, Steam Deck verified is a priority, no voice chat").
- **Per session:** add your instructions after the start prompt. They override the config; only the safety contract cannot be lifted.

## 2. Change the title

1. Choose a title and run a trademark search (legal/21\_marque-pi.md).
2. Ask the AI: "Replace the public title "300 Years Later" with "" everywhere it is public (READMEs, landing, Remotion, docs/design/70), without touching the codename CENTURY TEMPS or technical identifiers."
3. Check: python3 tools/privacy\_scan.py and the landing tests.

## 3. Change the game without coding

| You want to...                     | Edit                                                                                                  | Check                          |
|------------------------------------|-------------------------------------------------------------------------------------------------------|--------------------------------|
| add a transformation over time     | data/recipes/*.json (format: data/schemas/recipe.schema.json) + a Chronicle template in data/phrases/ | python3 tools/validate_data.py |
| add a contract                     | data/contracts/*.json                                                                                 | same                           |
| tune round length, speeds, scoring | data/tuning.json                                                                                      | playtest                       |
| change an era's look               | docs/design/33_VISUAL_TARGETS.md §2 then ask the AI to apply it                                       | visual review                  |
| add a trailer or landing shot      | data/shotlist.json                                                                                    | render + approval              |

## 4. Reuse the studio for another game

The skills (.claude/skills/) are 80 % generic: replace docs/design/ with your own design (keeping the same file structure), adapt CLAUDE.md and studio.config.yaml, then run game-studio. Keep the safety contract, decision gates and legal pack as a base.

# Using the kit with other AIs

 [Version française](#)

| Tool                                         | How                                                                                                                                                                              |
|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Claude Code                                  | native: skills in <code>.claude/skills/</code> , rules in <code>CLAUDE.md</code> , permissions and plugins in <code>.claude/settings.json</code> , Epic's official Unreal plugin |
| Gemini CLI                                   | reads <code>GEMINI.md</code> , which imports <code>AGENTS.md</code>                                                                                                              |
| GitHub Copilot                               | reads <code>.github/copilot-instructions.md</code> , which points to <code>AGENTS.md</code>                                                                                      |
| Kimi, Codex, Aider, Cursor, Windsurf, others | read <code>AGENTS.md</code> (otherwise send "Read <code>AGENTS.md</code> " as the first message), which points to the skills as Markdown procedures                              |
| Agent without access to the Unreal editor    | command-line editor Python: <code>UnrealEditor-Cmd.exe &lt;project&gt; -run=pythonscript -script=game/Scripts/&lt;script&gt;.py</code> and builds via <code>Build.bat</code>     |

## Universal prompt:

Read `AGENTS.md`, then apply the game-studio skill. Work autonomously from A to Z following `studio.config.yaml`; only come to me at hard stops, batching your questions with your recommendation. Reply in English. *(optional)* Extra instructions: ...

Whatever the tool: replicate the deny and ask rules of `.claude/settings.json` in its permission settings, and run `tools/privacy_scan.py` before every commit.

If the tool has no frontend-design skill, the landing strictly follows `docs/design/34_LANDING_CINEMATIQUE.md` and `30_ART_BIBLE.md`. If the tool does not support plugins, Epic's plugin is installed from Claude Code or replaced by editor Python.

# FAQ and troubleshooting

 [Version française](#)

**Does the repository contain game images?** No, on purpose. The game does not exist yet: every image will be rendered in Unreal Engine following `docs/design/33_VISUAL_TARGETS.md` and `data/shotlist.json`, then approved by a human. The diagrams in the repository explain how things work; they do not show the game.

**Does the AI really do everything on its own?** Almost: installation, code, tests, renders, landing, documents. It only stops where law, licences or your money require it: creating your accounts, installing Unreal from the launcher (Epic login), buying, signing, publishing, approving media, setting the price. It prepares everything so each of these is a few clicks, and keeps other tracks moving meanwhile. A game of this ambition also needs humans (playtesters, a creature artist, a composer, a lawyer).

**Can I add my own instructions?** Yes: in your message (takes priority) or in `extra_instructions` in `studio.config.yaml`. Only the safety contract cannot be lifted.

**Can the AI make a game on the level of GTA?** No. Those productions involve hundreds of people and budgets in the hundreds of millions. The kit aims at a very high-quality realistic indie game, with professionals for organic art (creatures, acting, music). See [03\\_TIME\\_AND\\_COST.md](#).

**How much does it cost?** From  $\approx$  €40,000 (solo + AI) to €500,000+ (small team), excluding the owner's pay. Details in guide 03.

**Can I sell the game I make with this kit?** Yes (MIT and CC BY 4.0 licences, with attribution). Respect the Unreal EULA, the Fab/MetaHuman licences, and choose your own title.

**The Unreal MCP server does not answer.** The editor must be open, the Model Context Protocol and AllToolsets plugins enabled, the server started (`ModelContextProtocol.StartServer`), and Git Bash on the PATH on Windows.

**Compilation is very slow.** Exclude project folders from the antivirus (carefully), enable a shared DDC cache, add RAM. Compile in Development Editor during development.

**privacy\_scan.py blocks my commit.** It found an email, a number, a token, a tracker or a term from your local list. Fix the reported file; do not bypass the check.

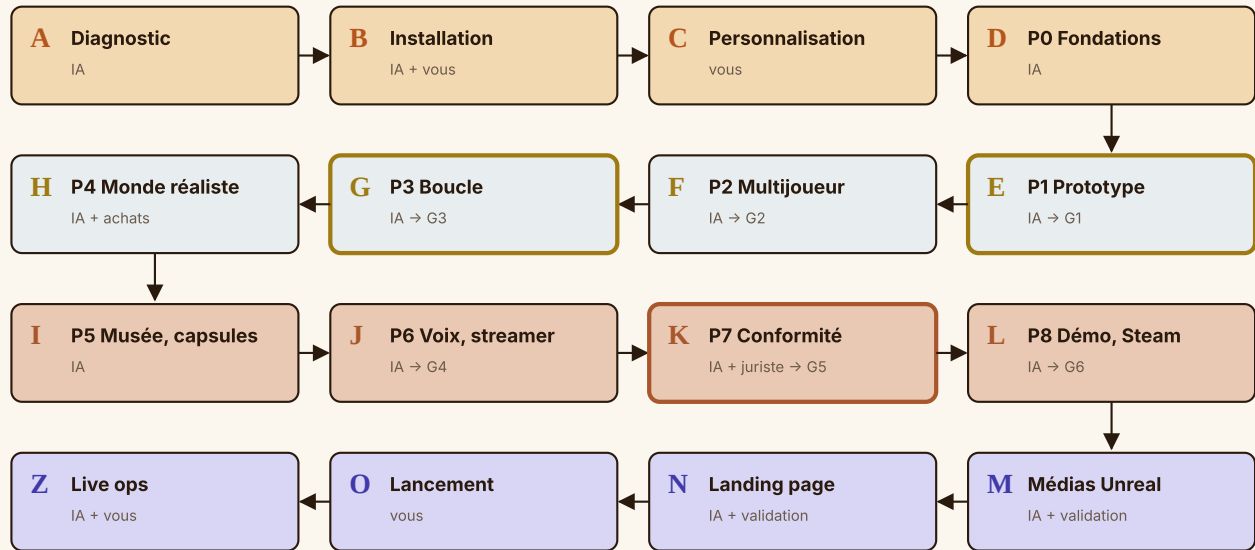
**Robot Framework tests fail locally.** Run `rfbrowser init` once after installation (downloads the test browsers).

**Can I use AI-generated images to go faster?** The kit forbids it by default: legal risk (rights), player perception, Steam disclosure duty. If you decide otherwise, record it in an ADR and disclose it on Steam and in `legal/17_transparence-ia.md`.

# Diagrams

## Le parcours du studio, de A à Z

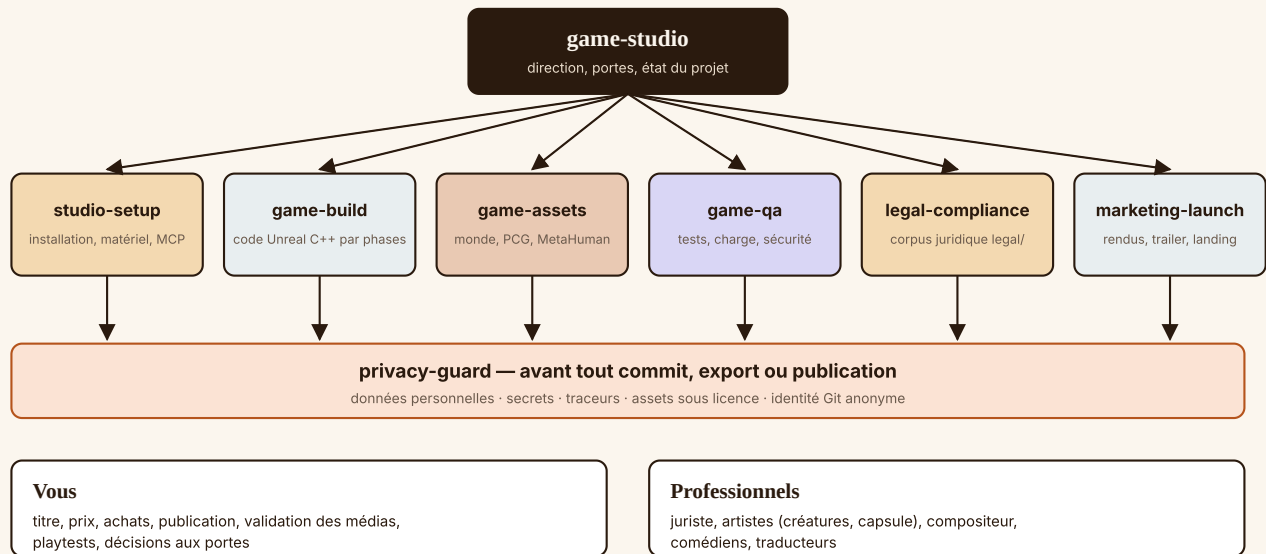
Les portes en couleur sont des arrêts obligatoires : playtest ou décision humaine.



parcours a z

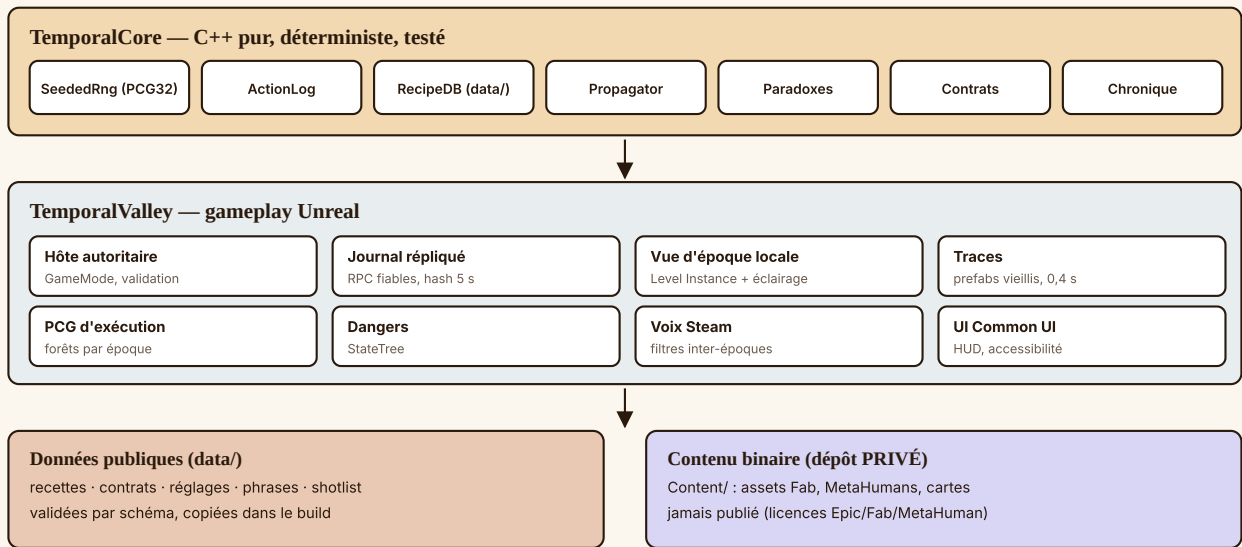
## Les skills : un studio en équipes

game-studio dirige ; chaque équipe a son skill ; privacy-guard contrôle tout ce qui sort.



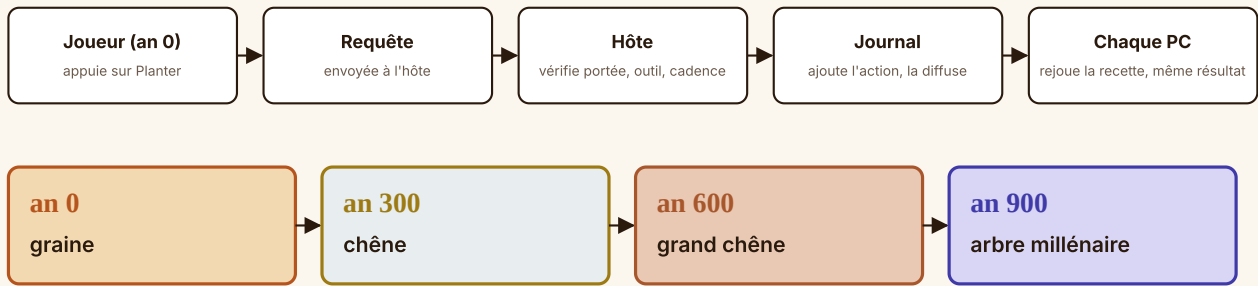
orchestration skills

# Architecture du jeu (Unreal Engine 5.8)



architecture unreal

## Ce qui se passe quand un joueur plante une graine



La graine de monde + le journal + les recettes suffisent à reconstruire les 4 époques : sauvegarde, capsule et musée utilisent le même journal.  
Si l'an 300 avait modifié l'arbre et que l'an 0 arrache la graine : non-conformité, la jauge monte, les Chronomites arrivent.

propagation

## Modèle de sécurité du kit

### L'IA peut

- ✓ lire et écrire dans le dépôt
- ✓ installer depuis les sources officielles, après accord
- ✓ compiler, tester, mesurer
- ✓ piloter l'éditeur Unreal sur une branche
- ✓ préparer documents, médias, landing page
- ✓ proposer achats, textes et décisions

### L'IA ne fait jamais seule

- × acheter, créer un compte, saisir un identifiant
- × publier (GitHub public, Steam, réseaux, site)
- × signer, déposer une marque, fixer le prix
- × lire ~/.ssh, .env, trousseaux, fichiers perso
- × ajouter traceur, cookie, CDN, télémétrie
- × publier Content/ ou des assets sous licence

Garde-fous : `.claude/settings.json (deny/ask)` · `tools/privacy_scan.py` · hook pré-commit · CI (`gitleaks`, `semgrep`, `ZAP`) · `media/APPROVALS.md`

Votre liste locale de termes interdits (`~/config/300yl/denylist.txt`) n'est jamais versionnée.

modele securite

# Screenshots

Real screenshots taken while generating this guide.

## Environment diagnosis (step A)

```
$ python3 tools/doctor.py
Système : Linux 6.18.44-fc-v77 – Python 3.13.16

✓ git      Versionnage           [tout] git version 2.43.0
✓ git-lfs  Contenu binaire (dépôt privé) [game-build] git-lfs/3.4.1 (GitHub; linux amd64; go 1.22.2)
✓ python3  Outils du kit                [tout] Python 3.13.16
x dotnet   .NET (UnrealBuildTool)        [game-build] MANQUANT → winget install Microsoft.DotNet.SDK.8
✓ node     Landing page / Remotion     [marketing-launch] v22.22.0
✓ npm      Paquets web                  [marketing-launch] 10.9.4
✓ ffmpeg   Encodage vidéo, GIF           [marketing-launch] ffmpeg version 6.1.1-3ubuntu5 Copyright (c) 2000-2023 the FF
x blender  Retouches d'assets            [game-assets] MANQUANT → https://www.blender.org/download/lts/
x robot    Tests E2E Robot Framework     [game-qa] MANQUANT → pip install robotframework robotframework-browser
x k6       Tests de charge web           [game-qa] MANQUANT → winget install GrafanaLabs.k6
x gitleaks Détection de secrets      [game-qa] MANQUANT → winget install Gitleaks.Gitleaks
x semgrep  Analyse statique sécurité     [game-qa] MANQUANT → pip install semgrep
✓ gh       Publication GitHub (par l'humain) [publication] gh version 2.89.0 (2026-03-26)
x unreal   Unreal Engine 5.8             [game-build] MANQUANT → Epic Games Launcher (voir studio-setup)
```

Les outils manquants sont installés par le skill « studio-setup ».

## Environment diagnosis (step A)

### Game data validation

```
$ python3 tools/validate_data.py
[validate_data] 25 recettes, 12 contrats, 16 modèles – jsonschema complet
✓ données valides
```

## Game data validation

### Privacy scan before publishing

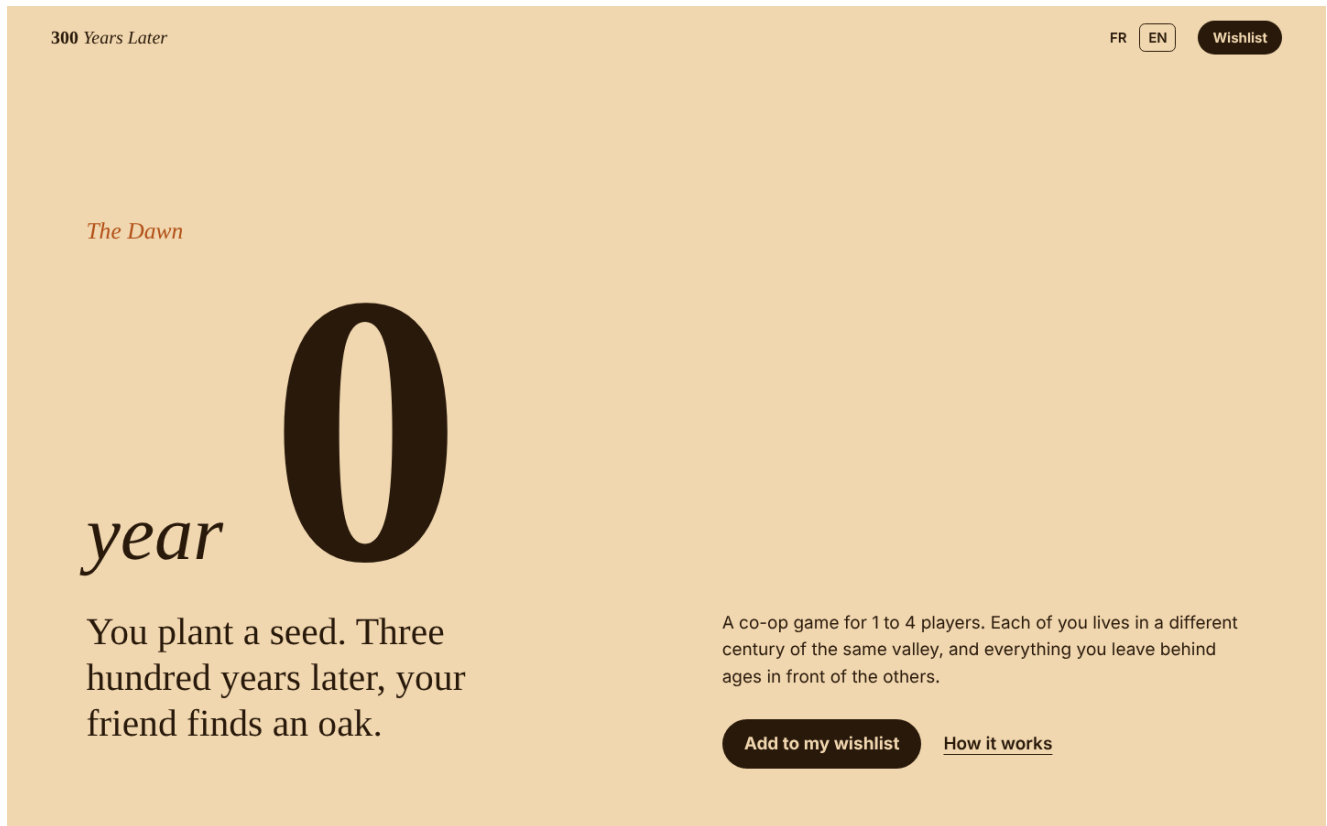
```
$ python3 tools/privacy_scan.py
[privacy_scan] 180 fichiers, 0 termes locaux, 0 problème(s)
  i Astuce : créez ~/.config/300yl/denylist.txt avec vos nom, e-mail, pseudo, employeur.
```

## Privacy scan before publishing

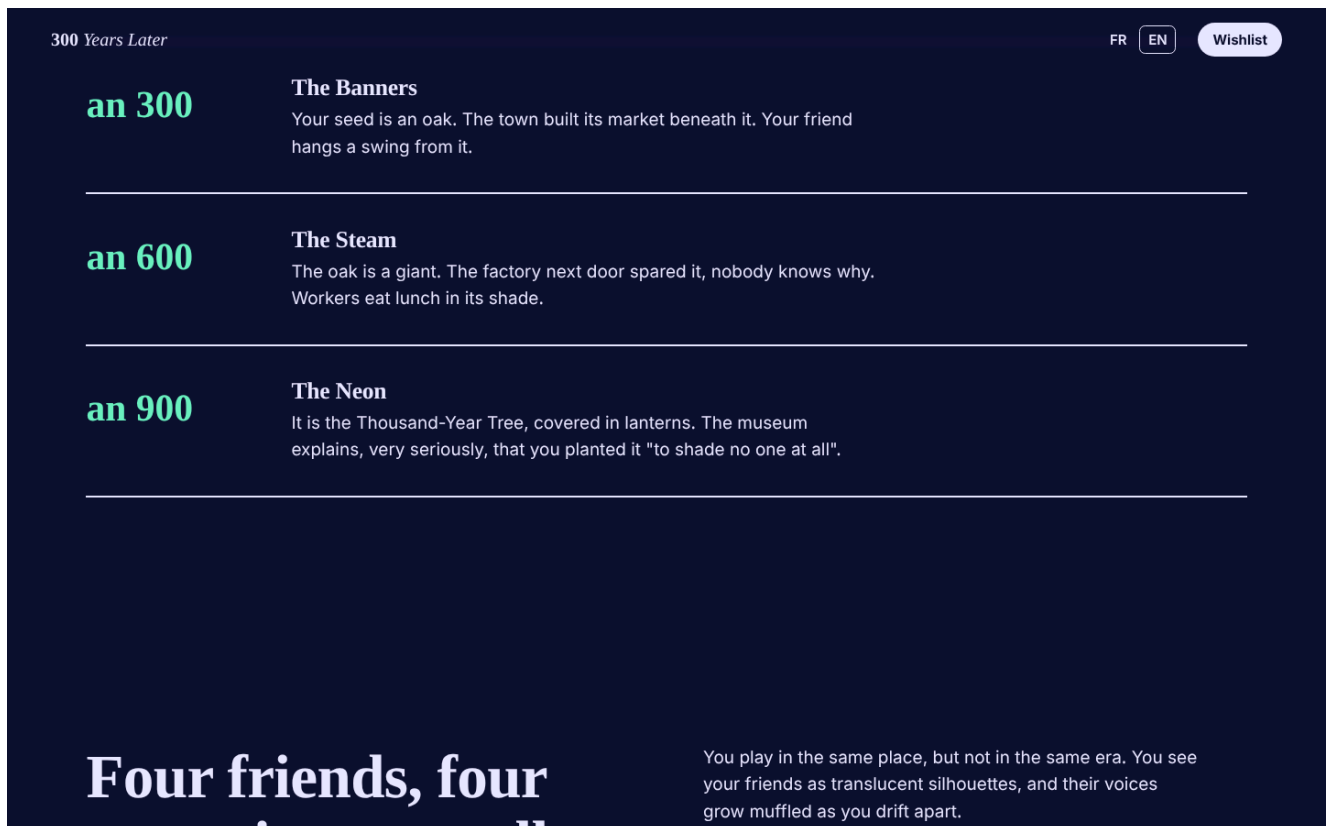
### Licence audit

```
$ python3 tools/license_audit.py
[license_audit] 0 dossier(s) vérifié(s), 0 problème(s)
```

## Licence audit



Reference landing page — desktop (typographic mode)



Landing — the page takes the colour of the era being read

## *The Dawn*

year **0**

You plant a seed. Three hundred years later, your friend finds an oak.

A co-op game for 1 to 4 players. Each of you lives in a different century of the same valley, and everything you leave behind ages in front of the others.

[Add to my wishlist](#)

[How it works](#)

# Game images

No approved render yet. The shots to produce are described in docs/design/33\_VISUAL\_TARGETS.md, docs/design/34\_LANDING\_CINEMATIQUE.md and data/shotlist.json; they will appear here automatically once marked approved in media/APPROVALS.md.